

Task 1 — QA & Debug Report

App: RealWorld "Conduit". Frontend `gothinkster/react-redux-realworld-example-app` (:4100). Backend `gothinkster/node-express-realworld-example-app` (Express + Prisma + PostgreSQL, :3000).

Tester: Sumit Goyal **Date:** 29 Sep 2026 **Method:** manual exploration and an automated Playwright run (`task1/qa-tests/qa.js`, 34 checks), then code review to confirm the cause of each bug.

Summary for stakeholders

The main user journeys work: sign up, log in, write, edit and delete articles, and log out. The app is **not ready for production**, for three reasons:

- 1. **Out of the box it can't reach any server.** It loads its data and its styling from public servers that no longer exist, so every screen is empty and unstyled. This alone explains most "it feels broken" reports.
- 2. **Account security is weak.** The app accepts a 1-character password and unlimited login guesses. Logging out doesn't really end the session, and a stolen token works for 60 days.
- 3. **Bad input isn't handled well.** Submitting an empty article crashes the whole app. Other bad input produces server errors or broken layouts, and users get no feedback.

14 issues are listed below, most severe first. Every one was reproduced and has evidence in `qa-tests/results.json` and `qa-tests/screenshots/`.

Bug table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	App depends on dead external servers: no data and no styling	Clone the frontend, <code>npm start</code> , open :4100	Expected: styled app, articles and login work. Actual: every API call to <code>conduit.productionready.io</code> fails, so the feed stays on "Loading...". The whole UI renders unstyled because <code>demo.productionready.io/main.css</code> returns 404. The public demo <code>demo.realworld.io</code> also returns 404.	Critical	API URL hardcoded in <code>src/agent.js:6</code> , stylesheet hotlinked from a third-party domain in <code>public/index.html:7</code> . No environment config and no uptime monitoring.
2	Session survives logout. The token stays valid for 60 days.	Log in, copy <code>jwt</code> from <code>localStorage</code> , click Logout, call <code>GET /api/user</code> with the old token	Expected: 401. Actual: 200 with the user's profile. The token decodes to <code>exp = 60 days</code> .	High	Stateless JWT with <code>expiresIn: '60d'</code> (<code>token.utils.ts:5</code>). Logout only clears <code>localStorage</code> on the client, and there's no server-side revocation.
3	Auth token stored in <code>localStorage</code>	Log in, then DevTools → Application → Local Storage	Expected: HttpOnly cookie. Actual: <code>jwt</code> key readable by any script on the page.	High	<code>middleware.js:52</code> <code>localStorage.setItem('jwt', ...)</code>
4	Weak passwords and invalid emails accepted	<code>POST /api/users</code> with password <code>1</code> , or email <code>notanemail</code>	Expected: 422 with a message. Actual: 201, account created.	High	<code>auth.service.ts</code> only checks for blank values. No length, strength or format validation on client or server.

5	No brute-force protection on login	Send 20 wrong-password logins in a row	Expected: lockout or HTTP 429. Actual: all 20 get 403. Unlimited guessing.	High	No rate-limit middleware (e.g. <code>express-rate-limit</code>)
6	Malformed input returns HTTP 500 and leaks internals	(a) POST <code>{bad json}</code> to <code>/api/users/login</code> (b) GET <code>/api/articles?limit=100000&offset=-5</code>	Expected: 400 Bad Request. Actual: 500 with the raw parser/Prisma error message.	High	Global error handler in <code>main.ts</code> maps every unknown error to 500 and echoes <code>err.message</code> . <code>offset/limit</code> are passed to Prisma unvalidated (<code>article.service.ts:82-83</code>).
7	Logged-out users can open the editor. Other users can open <i>my</i> article's editor.	(a) Log out, go to <code>/editor</code> (b) as user B, open <code>/editor/<userA-slug></code>	Expected: redirect to login / "not allowed". Actual: the form is shown, prefilled with A's content. The API correctly rejects the save with 403, but only after the user has done the work, and no error is shown.	Medium	No route guards in <code>App.js</code> . Authorization exists only on the API.
8	No length limits: a 10,000-character title breaks the layout	Create an article with a 10,000-character title	Expected: max-length validation. Actual: 201, 10,009-character slug/URL, page scrolls horizontally.	Medium	No max length in the form, API or DB schema. Title has no CSS <code>word-break</code> .
9	Publishing an empty (or duplicate-title) article crashes the app	New Post → click "Publish Article" with nothing filled in	Expected: "Title can't be blank". Actual: the API correctly returns 422, but the frontend crashes with <code>TypeError: Cannot read properties of undefined (reading 'slug')</code> . In production this is a blank white screen.	High	<code>reducers/common.js:40</code> reads <code>action.payload.article.slug</code> without checking <code>action.error</code> . It assumes every submit succeeds.
10	Article deleted with no confirmation	Open own article → "Delete Article"	Expected: "Are you sure?" Actual: permanently deleted instantly (API 404 afterwards).	Medium	Delete handler calls the API directly, with no confirm or undo
11	Unknown article URL shows a blank page	Visit <code>/article/does-not-exist</code> , or press Back after deleting an article	Expected: "Article not found" page. Actual: empty page with only the navbar.	Medium	No 404 route or empty state. The Article component renders nothing when the fetch fails.
12	Editing a title silently breaks old links	Edit an article's title, then open the old URL	Expected: old link redirects. Actual: slug changes, old URL returns 404 (blank page).	Low	Slug is regenerated from the title on update, and no redirect is kept
13	Accessibility: form inputs have no labels. Mobile layout overflows.	Inspect <code>/register</code> . View home page at 375px width.	Expected: labelled inputs, no horizontal scroll. Actual: 3/3 inputs have only placeholders, and the page overflows horizontally at 375px.	Low	Template uses <code>placeholder</code> instead of <code><label> / aria-label</code> . No responsive handling.
14	Missing security hardening and a failing seed script	Inspect API response headers. Run <code>npx prisma db seed</code> .	Expected: CORS limited to the app origin, no CSP, X-Powered-By hidden. Actual: <code>Access-Control-Allow-Origin: *</code> , no CSP / X-Frame-Options, X-Powered-By: Express. Seed crashes with <code>P2002 Unique constraint failed on name</code> .	Low	<code>app.use(cors())</code> with defaults, no <code>helmet</code> . Seed uses <code>create</code> with random faker tag names that collide.

Checked and working: stored XSS in article markdown (sanitised), duplicate email message, API ownership checks on PUT/DELETE (403), double-submit on slow network (button disabled), empty comments rejected, invalid JWT returns 401. Average `GET /articles` latency is about 570 ms against the remote DB.

Root-cause analysis — Issue #2: the session survives logout

When a user logs in, the API signs a JSON Web Token with `expiresIn: '60d'` and the frontend keeps it in `localStorage`. The server keeps no record of issued tokens. It only checks each token's signature and expiry, so any token it signed is accepted until its expiry date. "Logout" in the React app simply runs `localStorage.setItem('jwt', '')`. That removes the browser's copy but does nothing on the server. We showed this by calling `GET /api/user` with the token after logging out: it still returned **200** and the user's profile. So if a token is ever leaked (through XSS, a shared computer, a browser extension or logs), the attacker can use it for up to 60 days, and neither the user logging out nor changing their password stops it. Issue #3 makes this worse, because the token sits in storage any script can read. To fix it: (1) use short-lived access tokens (about 15 min) plus a refresh token stored in the database; (2) add a `POST /api/users/logout` endpoint that revokes the refresh token, or keep a `tokenVersion` on the user row that is bumped on logout or password change and checked in the auth middleware; (3) deliver tokens in an `HttpOnly; Secure; SameSite=Strict` cookie instead of `localStorage`. The Playwright check `O1` then serves as a regression test.

Evidence



#2 After logout. The old token still returned HTTP 200 (qa-tests/results.json → O1).



#7 User B sees user A's article in the editor

